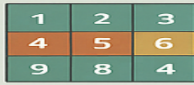


NUMPY CONCEPTS

1D ARRAY



2D ARRAY



3D ARRAY



ndarray

VIEWING & MODIFYING



ARRAY MANIPULATIONS



DOT PRODUCT & MATRIX MULTIPLICATION

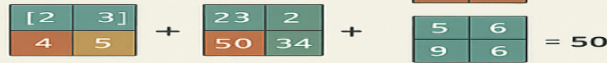
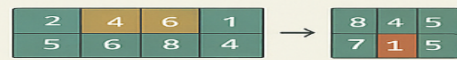


IMAGE TO ARRAY



IMAGE TO ARRAY



Python

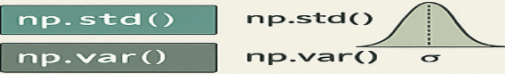
DATA TYPES & ATTRIBUTES

dtype	np.int32
shape	(3, 3)
size	9
ndim	proprecuif

ARRAY MANIPULATIONS



STATISTICAL FUNCTIONS



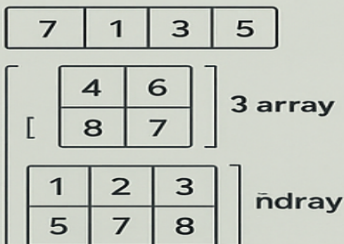
SORTING & FILTERING



SORTING & FILTERING



1D, 2D and 3D Arrays

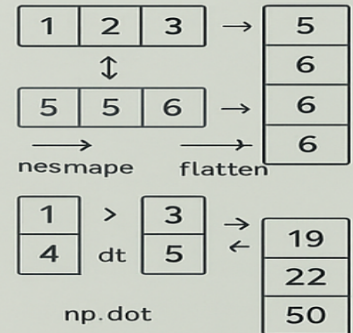


dtype	shape
size	ndim

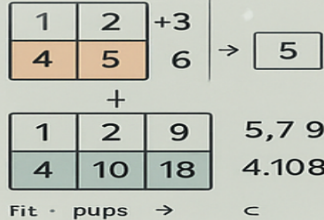
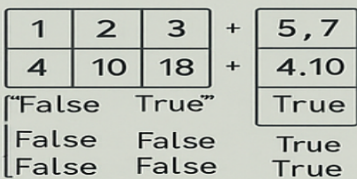
Random

np.random.rand()
np.random.randint
np.random.seed()

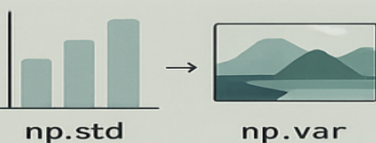
Array Manipulation



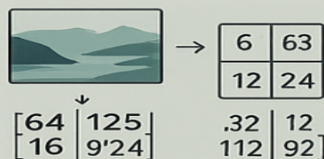
Element-wise Operations



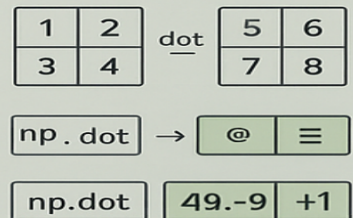
Statistical Functions



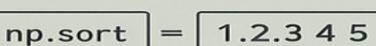
Images as Arrays



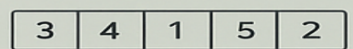
Dot Product and Matrix Multiplication



Images as Arrays



Sorting and Filtering



Why numpy? Numerical Python.

↳ similar to python lists.

Then why numpy?

→ cuz python lists are slow

→ machines usually understand only 1's & 0's and things like numbers which can be contained in numpy are easy for machines to understand versus something more complicated.

Ex eg: if a patient has heart disease instead of yes or no, we convert it to one or zero

→ This idea of converting data into something that machine can understand, like numbers like binary allows us to make machines learn

→ Machines learn by performing multiple operations and by storing this information in an array like numpy.

→ behind the scenes optimizations written in C

→ Vectorization via broadcasting (to avoid loops)

Things to begin with,

→ most useful functions

→ Numpy datatypes & attributes (ndarray)

→ creating arrays

→ viewing arrays and matrices

→ Manipulating & comparing arrays

→ Use cases

Data types and Attributes

ndarray $\rightarrow$ n dimensional array

```
a1 = np.array([1, 2, 3])
```

a1

```
array([1, 2, 3])
```

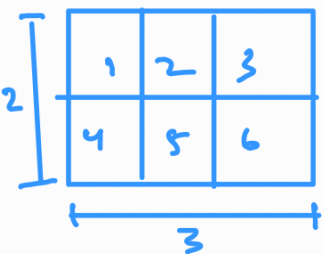
```
a2 = np.array([[1, 2, 3],  
               [4, 5, 6],  
               ])
```

a2

```
>> array([[1, 2, 3],  
          [4, 5, 6]])
```

Anatomy of a Numpy array

Data



Numpy

```
array([1, 2, 3])
```

```
array([[1, 2, 3],  
       [4, 5, 6]])
```

Details

Names: Arrays, vectors

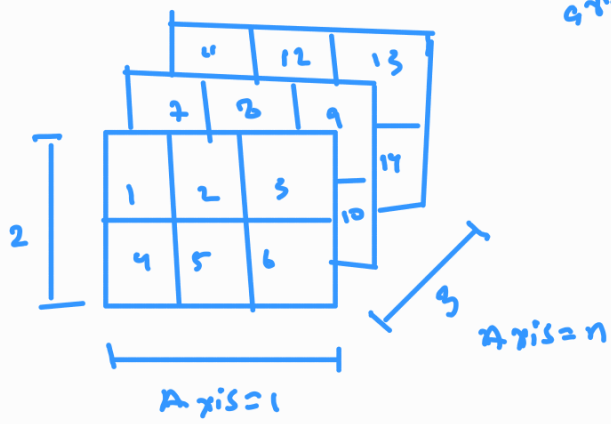
1-dimensional

shape = (1, 3)

Names: Arrays, vectors

2-dimensional

shape = (2, 3)



```
array([[
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]]
    [
    [10, 11, 12],
    [13, 14, 15],
    [16, 17, 18]]])
```

Names: Arrays, vectors
 more than 1 dimension
 shape = (2, 3, 3)

Datatypes and Attributes:

```
import numpy as np
```

i) `a1 = np.array([1, 2, 3])`

`a1`

```
>> array([1, 2, 3])
```

ii) `a2 = np.array([[1, 2, 3], [4, 5, 6]])`

`a2`

```
>> array([[1, 2, 3],
          [4, 5, 6]])
```

iii) `.shape`

`a1.shape`

```
>> (3,)
```

`a2.shape`

```
>> (2, 3)
```


iv) .ndim

a1.ndim, a2.ndim

>> (1, 2)

vi) .type

a1.type, a2.type

>> (dtype('int64'), dtype('int64'))

vii) .size (number of elements present)

a1.size

>> 3

a2.size

>> 6

viii) type()

type(a1), type(a2)

>> (numpy.ndarray, numpy.ndarray)

Create a dataframe from numpy array

```
a2 = np.array([[1, 2, 3],  
               [4, 5, 6]])
```

```
import pandas as pd
```

```
df = pd.DataFrame(a2)
```

```
df
```

	0	1	2
0	1	2	3
1	4	5	6

Creating numpy arrays

→ Sample-array = np.array([1, 2, 3])

Sample-array

```
>> array([1, 2, 3])
```

→ ones = np.ones() (shape, dtype = None, order = 'C')

ones = np.ones((2, 3))

ones

```
>> array([[ 1.,  1.,  1.],
          [ 1.,  1.,  1.]])
```

→ ones.dtype

```
>> float64
```

→ type(ones)

```
>> numpy.ndarray
```

→ range-array . arange(start, stop, step)

range-array = np.arange(0, 10, 2)

→ random int array

random-array = np.random.randint(0, 10, size=(2, 5))

random-array

```
>> array([[ 1,  3,  0,  2, 10, 12],
          [ 6, 14,  6, 11,  6,  1]], dtype=int32)
```

→ .shape

random-array.shape

>> (2, 5)

→ random-array2 = np.random.random(size=(2, 4))

random-array2

```
>> array([[ 0.6181,  0.0681,  0.9155,  0.1212],
          [ 0.473 ,  0.5321,  0.1234,  0.3456]])
```

→ random-array3 = np.random.rand(d0, d1, d2...dn)

→ np.random.seed() (pseudo random numbers)

Numpy Random seed()

↳ prevents the generated random numbers from changing when re-run or shared

```
np.random.seed(0)
```

```
random_array = np.random.randint(10, size=(2,2))
```

random array

```
>> array([[5, 2],  
          [9, 5]], dtype=int32)
```

Viewing arrays and matrices

① unique

```
df = np.array([[1,2,3], [3,4,5]])
```

```
np.unique(df)
```

```
>> array([1,2,3,4,5])
```

② accessing values

```
df[0]
```

```
>> array([1,2,3])
```

```
df[1:3][0]
```

```
>> array([3,4,5])
```


Manipulating Arrays

i) addition (or) `np.add(a, b)`

`a = [1, 2, 3]`

`b = np.ones(3)` // `[1, 1, 1]`

`a + b`

`>> array([2., 3., 4.])`

ii) subtraction (or) `np.subtract(a, b)`

`a - b`

`>> array([0, 1, 2])`

iii) multiply (or) `np.multiply(a, b)`

`a * b`

`>> array([1, 2, 3])`

iv) divide (or) `np.divide(a, b)`

`a / b`

`>> array([2, 2, 3])`

⊕ how this works?

The smaller array is "broadcasted" across the larger array so that they have compatible shapes.

Aggregation.

↳ formation of a number of things into a cluster.

→ `a1 = np.array([1,2,3])`

```
sum(a1)
>> 6
np.sum(a1)
>> 6
```

So when to use when?

use python methods on py datatypes
and numpy methods on numpy arrays

eg.

massive-array = `np.random.random(10000)`

`%timeit sum(massive-array)` *# py sum()*

`%timeit np.sum(massive-array)` *# numpy sum()*

→ `607 μs ± 28.8 μs per loop (mean ± std. dev. of 7 runs, 1000 loops each)`

→ `6.16 μs ± 673 μs per loop (mean ± std. dev. of 7 runs, 100,000 per loop each)`

→ `np.mean(a1)`

→ 2

→ `np.std(a1)` → a measure of how spread out of a group of numbers is from the mean

→ 0.81

→ `np.max(a1)`

→ 3

→ `np.min(a1)`

→ 1

→ `np.var(a1)` → (measure of the avg degree to which each number is different to the mean)

→ 0.66

higher variance = wider range of numbers
low variance = lower range of numbers

$$\text{std} = \sqrt{\text{var}}$$

Standard deviation and variance.

↳ `np.std()`

↳ `np.var()`

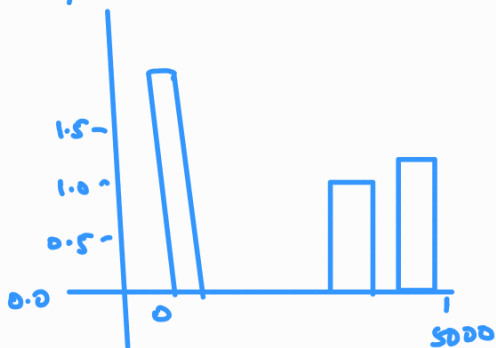
```
high-ver-err = np.array([1, 100,
                        200, 300, 4000, 5000])
```

%.matplotlib inline

```
import matplotlib.pyplot as plt
```

```
plt.hist(high-ver-err)
```

```
plt.show()
```



Reshape and Transpose.

```
a 32 np.random.randint(0, 10, size = [2, 3, 8])
```

```
a3
```

```
>> array([[[[ 5, 0, 2],
             [ 3, 7, 1],
             [ 3, 5, 2]],
           [[ 4, 7, 1],
             [ 8, 2, 1],
             [ 6, 7, 7]]], dtype=int32)
```

a2

```
>> array([[1, 2, 3],  
          [4, 5, 6]])
```

$a2 * a3 \rightarrow$ operands could not be broadcast together with
shapes (2,3) (2,3,3)

`a2reshaped = a2.reshape(2,3,1)`

`a2reshaped`

```
>> array([[[1],  
          [2],  
          [3]]  
        [[4],  
          [5],  
          [6]]])
```

(while reshaping
dimensions must be
equal or set to 1)

Now, `a2reshaped * a3`

```
>> array([[[4, 0, 2],  
          [6, 14, 18],  
          [9, 15, 6]]],  
        [[16, 28, 24],  
          [40, 40, 5],  
          [36, 42, 42]]])
```

Transpose (rows to cols & vice versa)

eg
a2
array([[1, 2, 3],
 [4, 5, 6]]) | shape
 (2, 3)

a2.transpose() (or) a2.T → shorthand

→ array([[1, 4],
 [2, 5],
 [3, 6]]) | shape
 (3, 2)

Why would we use Transpose and when would we?

→ for dot product of 2 arrays

Numpy multiplication.

i) element wise (hadamard product)

ii) dot product

i) element wise product

np.random.seed(0)

r1 = np.random.randint(0, 10, size=(5, 2))

r2 = np.random.randint(0, 10, size=(5, 3))

np.multiply(r1, r2)

$x_1 = \begin{bmatrix} 5, 0, 3 \\ 3, 7, 9 \\ 3, 5, 2 \\ 4, 7, 6 \\ 8, 8, 1 \end{bmatrix}$, $x_2 = \begin{bmatrix} 6, 7, 7 \\ 8, 1, 5 \\ 9, 2, 9 \\ 4, 3, 0 \\ 3, 5, 0 \end{bmatrix}$

`np.multiply(x1, x2)`

$\Rightarrow \begin{bmatrix} 30, 0, 21 \\ 24, 7, 45 \\ 27, 40, 18 \\ 16, 21, 0 \\ 24, 40, 0 \end{bmatrix}$

ii) Dot product

`np.dot(x1, x2)` $\rightarrow$ error dimension mismatch
 $\swarrow \quad \searrow$ (shapes (5,2) and (5,2) not aligned, 3(dim1) != 5(dim0))
 $5 \times 3 \quad 5 \times 2$ for dot product it inner values should be

same and outer value is the resultant shape of matrix

so, we have to transpose

$\rightarrow \text{np.dot}(x_1, x_2.T)$ $(5,3) (3,5)$

`array([[51, 55, 72, 20, 15],
 [30, 76, 44, 33, 44],
 [67, 39, 85, 27, 34],
 [115, 69, 146, 37, 47],
 [111, 77, 145, 56, 67]], dtype=int32) (5x5)`

Break down

element wise

A	B
C	D

 2×2

E	F
G	H

 2×2 $\rightarrow$

A^*E	B^*F
C^*G	D^*H

Dot product

$r \times c$ $r \times c$
should be
same

A	B	C
D	E	F
G	H	I

 3×3

$\cdot \text{dot}$

J	K
L	M
N	O

 3×2

match

(3x2) shape

 $\rightarrow$

$AJ + BL + CN$	$AK + BM + CO$
$DJ + EL + FN$	$DK + EM + FO$
$GJ + HL + IN$	$GN + HM + IO$

Comparison Operators

- i) $>$
- ii) $<$
- iii) $==$
- iv) $>=$
- v) $<=$
- vi) $!=$

Sorting Arrays

```
random_array = np.random.randint(10, size=(3,5))  
random_array
```

```
>> array([[8, 1, 1, 3, 9],  
         [9, 3, 6, 7, 2],  
         [0, 3, 5, 9, 4]])
```


→ `np.sort (random-array)`

```
>> array([ [ 1, 1, 7, 8, 9 ],  
          [ 2, 3, 6, 7, 9 ],  
          [ 0, 3, 4, 5, 9 ] ], dtype=int32)
```

→ `np.argsort (random-array)`
 ↳ sorts index

```
>> array([ [ 1, 2, 3, 0, 4 ],  
          [ 4, 1, 2, 3, 0 ],  
          [ 0, 1, 4, 2, 3 ] ])
```

→ `argsmin()`

→ `argsmax()`

axis
↓ 0
→ 1

Images to Numpy Arrays.

from `matplotlib.image` import `imread`

```
>> dog = imread("file path")
```

```
dog.size, dog.shape, dog.ndim
```

```
>> (993600, (432, 575, 4), 3)
```

```
print (type(dog))
```

```
>> <class numpy.ndarray>
```

